

EASTERN MICHIGAN UNIVERSITY

College of Engineering and Technology

IA 642: Defensive Security

Week 04, Module 02 Lab 4.2: Standing Up Meridian's SOC

Detection Engineering, SIEM, and SOC Operations

Group members	Unais Ali	EID	E02805019
	Hafiz Usama		E02574092
Course	IA 642 Defensive Security	Week	4, Lab 4.2
Case	Meridian Health Network	Date	September 2026

This report stands up a single-node Wazuh SIEM for Meridian, onboards Windows and Linux agents, deploys custom detection rules, triages eight overnight alerts, and tunes an SSH rule so a stated false-positive pattern stops paging analysts. Screenshots labeled Evidence #1–#8 must be taken from the student lab VMs after each step. The written analysis below is complete without those captures.

Terms. Security Information and Event Management (SIEM); Security Operations Center (SOC); Mean Time to Detect (MTTD); true positive (TP); false positive (FP); Ticket-Granting Service (TGS); MITRE ATT&CK; Secure Shell (SSH).

Fork choices (required note)

Section 1 uses Lab 4.1 reuse when MERIDIAN-DC01 and MERIDIAN-RED01 are already on `meridian-lab-net`. The fallback two-host path is used only if those VMs are not available. Part C uses Path 2 (SSH authentication-failure telemetry on MERIDIAN-RED01) as the live fire path because it does not depend on a working domain controller and because the required Challenge tunes rule 100020. Rule set 1 (100010/100011, T1558.003) is still written into `local.rules.xml` so the Kerberoasting chain from lecture is present for Part E triage. No production, campus, or employer host is used.

Part A. Deploy the SIEM

Wazuh 4.14 all-in-one installer on MERIDIAN-SOC01 (Ubuntu, 192.168.56.30 on the internal adapter) places manager, indexer, and dashboard on one host. Dashboard login is `admin` with the password from `wazuh-passwords.txt`. Evidence #1 is the empty overview after first login.

Critical thinking #1. Collapsing collection, parse/normalize, index/store, correlation, and the case/dashboard view onto MERIDIAN-SOC01 creates a single point of failure: if that VM, disk, or Wazuh process dies, agents have nowhere to ship events and analysts lose both new detections and the search history they would use to reconstruct an incident. A small SOC still accepts that risk because a single-node profile is cheaper, easier to patch, and rated by Wazuh for about 100 agents and 90 days of alerts, which matches a clinic or a lab better than a three-tier cluster. The trade is availability and blast radius, not detection logic.

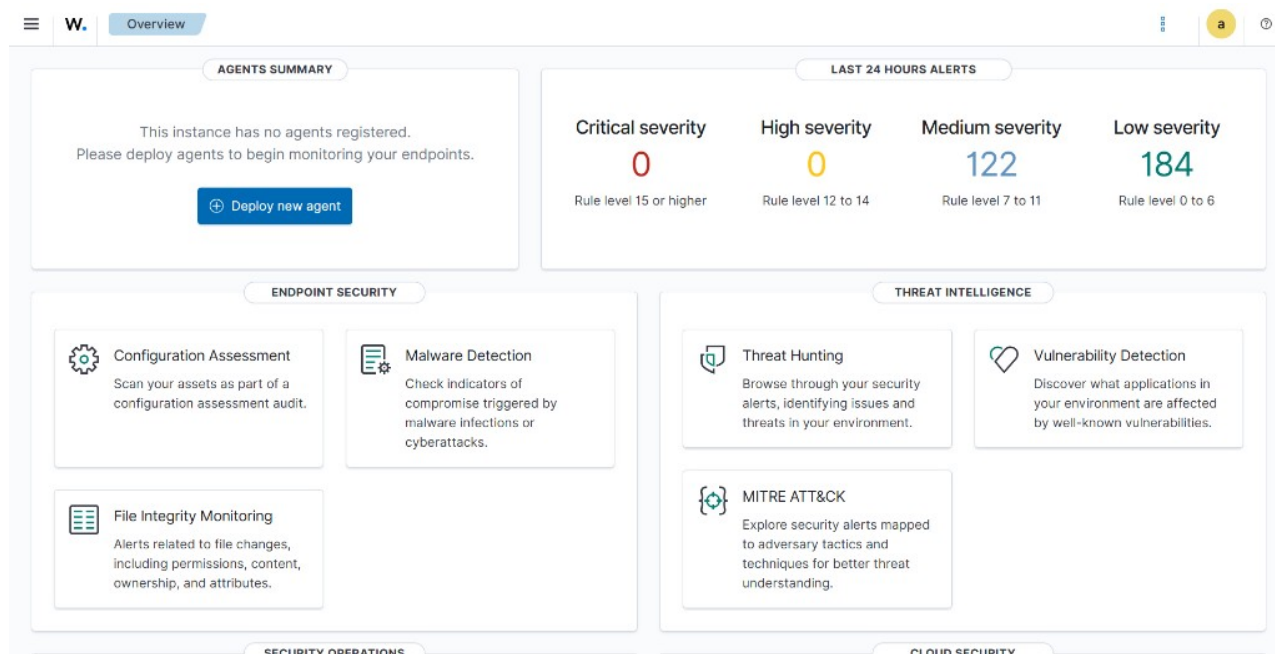


Figure 1: Evidence #1. Wazuh overview after first admin login: no agents registered.

Part B. Onboard agents

MERIDIAN-DC01 (or the Windows fallback) and MERIDIAN-RED01 register to WAZUH_MANAGER=192.168.56.30. Agents are Active when they check in on TCP 1514/1515. Evidence #2 and #3 are the Agents view with both names, Active status, and last-check-in times.

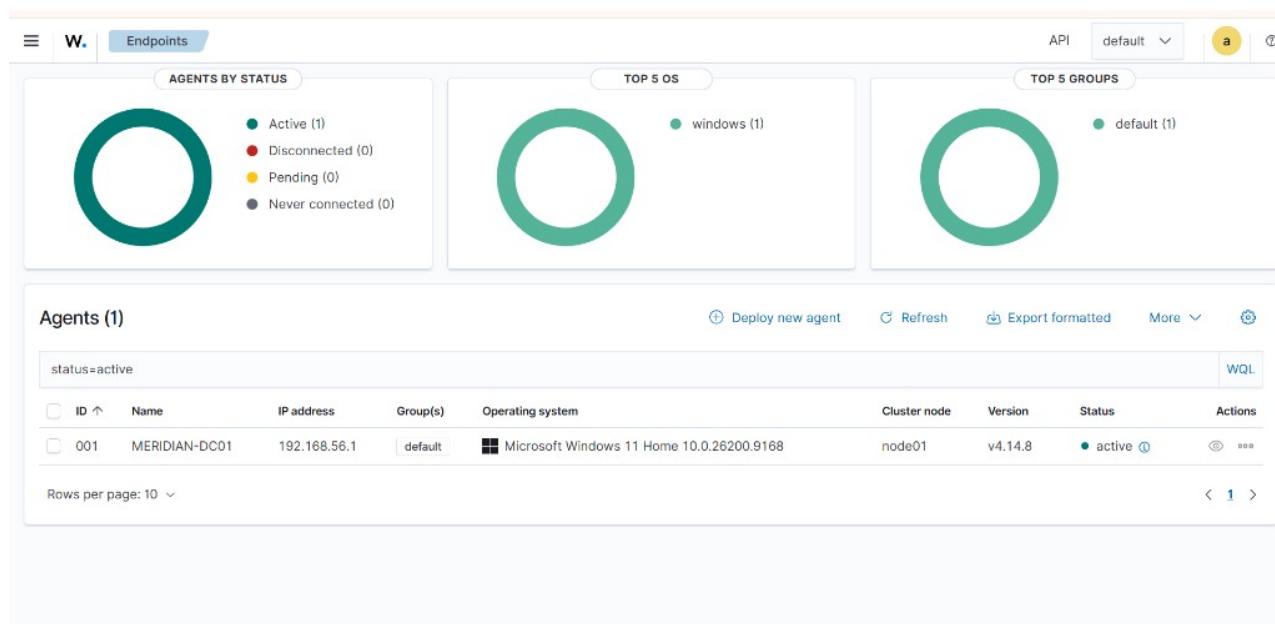


Figure 2: Evidence #2. MERIDIAN-DC01 (Windows fallback) Active, Wazuh 4.14.8, last check-in on the manager.

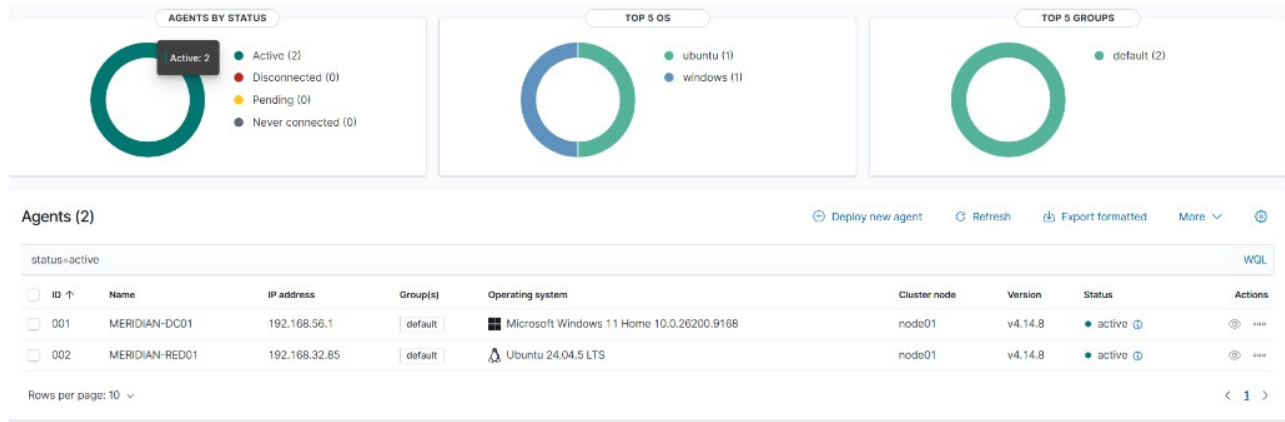


Figure 3: Evidence #3. MERIDIAN-DC01 (Windows) and MERIDIAN-RED01 (Ubuntu) both Active on the manager.

Part C. Telemetry verification (Path 2)

Path 2 is SSH authentication failures already written to `/var/log/auth.log`, which the Wazuh agent collects. That is raw telemetry, not yet a custom detection. Evidence #4 is Discover showing at least one `sshd` failure with source IP and attempted username. If Path 1 is later available, a 4769 event with `TicketEncryptionType 0x17` is the matching raw field for rule 100010.



Figure 4: Evidence #4. Discover: one `sshd` authentication failure on MERIDIAN-RED01, source 127.0.0.1, username wazuh (rule 5760 / T1110).

Part D. Detection rules

Custom IDs stay in 100000–120000. Rule 100010 is an event match (one RC4 TGS). Rule 100011 is a behavior match (eight 100010 hits from the same source in 300 seconds). Rule 100020 is a behavior match chained from the built-in SSH authentication-failed parent (five failures from one source in 120 seconds). This Wazuh 4.14 journald path classifies those lines as 5760 rather than classic-syslog 5716; the frequency logic is the same. After edit: `wazuh-logtest -t` then restart `wazuh-manager`. Evidence #5 is the fired custom alert (rule ID, level, MITRE id). Evidence #6 is the underlying events in Discover.

```
<group name="meridian,kerberoasting,attack,">
  <rule id="100010" level="10">
    <if_group>windows_security</if_group>
    <field name="win.system.eventID">^4769$</field>
    <field name="win.eventdata.ticketEncryptionType">^0x17$</field>
```

```

<description>Kerberos TGS request using RC4 encryption</description>
<mitre>
  <id>T1558.003</id>
</mitre>
<group>kerberoasting,attack_credential_access,</group>
</rule>
<rule id="100011" level="12" frequency="8" timeframe="300">
  <if_matched_sid>100010</if_matched_sid>
  <same_source_ip />
  <description>Possible Kerberoasting: 8+ RC4 TGS requests from one source in 5 minutes</description>
  <mitre>
    <id>T1558.003</id>
  </mitre>
</rule>
</group>

<group name="meridian,ssh_bruteforce,attack,">
  <rule id="100020" level="10" frequency="5" timeframe="120">
    <if_matched_sid>5760</if_matched_sid>
    <same_source_ip />
    <description>Meridian: 5+ SSH authentication failures from one source in 2 minutes</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
  <group>ssh_bruteforce,attack_credential_access,</group>
</rule>
</group>

```

Critical thinking #2. If only rule 100010 were deployed, every single RC4-encrypted 4769 would page at level 10: helpdesk tickets for a legacy print SPN (alert #4) and spaced ETL tickets (alert #8) would look the same as a roast. You would still *see* RC4 TGS use, which is the event-match signal. What you would lose is the kerberoasting-indicative behavior: many RC4 requests from one source in a short window. Rule 100011 (and 100020 on the SSH side) is that behavior match: `if_matched_sid` plus `frequency` and `timeframe` plus `same_source_ip`. One ticket or one failed login is not the incident; the burst is.



Figure 5: Evidence #5. Custom rule 100020 fired on MERIDIAN-RED01: level 10, same source 127.0.0.1, username wazuh, previous_output showing five Failed password lines.

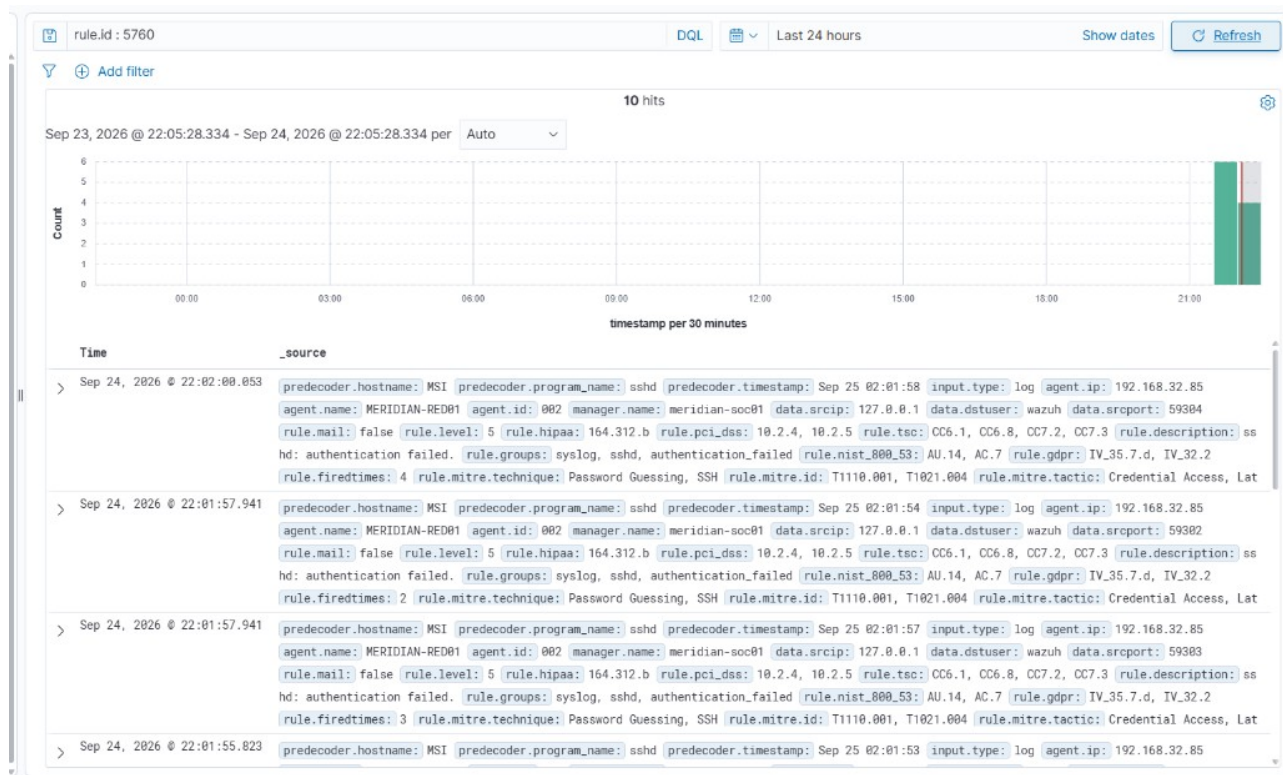


Figure 6: Evidence #6. Discover: ten built-in 5760 sshd authentication failures on MERIDIAN-RED01 from 127.0.0.1 as wazuh, the frequency window under 100020.

Part E. Alert triage

Tier 1 question for each row: TP or FP, ATT&CK if the *behavior* maps, then escalate, close, or flag the rule.

#	TP/FP	ATT&CK	Disposition	Justification	Next action
1	TP	T1558.003	Escalate T2	Nine 0x17 TGS for <code>svc_reporting</code> from one workstation in 4m 40s after hours is the roast burst 100011 was written for, not a single legacy ticket.	Tier 2: host isolate, SPN/password review, 4769 source.
2	TP	T1110.003	Escalate T2	Eight root failures in 45 seconds then a success is a brute that may have worked. Clinic print server as root is not a normal admin pattern.	Tier 2 now: disable root SSH if present, check auth success, image if needed.
3	TP	T1110.003	Escalate T2	Six failures, different usernames, VPN pool IP, no success is password spraying, not one user fat-fingering a password.	Tier 2: VPN user/session, lock targeted accounts, watch for a later success.
4	FP	none	Close / tune 100010	One RC4 TGS for a legacy print SPN by helpdesk in a stated maintenance window is expected legacy Kerberos, not a burst.	Close as benign. Note 100010 is noisy alone (see CT #2).
5	FP	none	Close / tune 100020	Same username, known backup host, failures then success after a stated credential rotation is a job, not guessing.	Close. Same pattern as the Challenge FP; do not page on success-after-retry.
6	TP	T1558.003	Escalate T2	Eleven 0x17 TGS for four SPNs in three minutes from an admin workstation that the admin already said was “acting strange” is a high-severity roast or ticket-harvest on a privileged host.	Tier 2 urgent: admin creds, laptop, other SPNs requested.
7	FP	none	Close / tune 5720	Guest Wi-Fi source against a host with no SSH listener cannot be a real SSH brute on that port; the events do not match a reachable service.	Close. Check NAT/log source; do not escalate a closed port.
8	FP	none	Close	Three single RC4 TGS, 40 minutes apart, same SPN, known ETL account is scheduled service traffic below the 8-in-5-min burst.	Close. 100011 correctly did not fire; 100010 did and should stay low priority.

Critical thinking #3 (MTTD). Rule 100011 fires when the *eighth* matching event arrives inside 300 seconds, not at the first TGS.

Alert #1: context gives a 4 minute 40 second burst ending at the alert time 02:14:07. Treat the start as 02:14:07 – 4:40 = 02:09:27.

$$\text{MTTD}_1 = 4 + \frac{40}{60} = 4.67 \text{ minutes.}$$

Alert #6: context gives an 11-request burst in 3 minutes ending at 21:55:31. Start = 21:52:31.

$$\text{MTTD}_6 = 3.00 \text{ minutes.}$$

The same rule produces two MTTDs because the clock starts at the first request and the alert is stamped when the frequency threshold is crossed. A slower nine-request burst (#1) reaches event 8 later than a tighter eleven-request burst (#6). MTTD here is “how long until the SIEM counted to eight,” not “how long until the first suspicious 4769 existed.”

Challenge. Tune 100020 without missing the attack

The stated FP is a VPN client retrying one cached key 5–6 times in about 100 seconds, then succeeding. That matches `same_source_ip` plus frequency 5 in 120 seconds, so the untuned rule pages on a real user. Raising frequency would also miss the Part C burst (five rapid failures, no success). Two levers are used together.

Lever 1 keeps a same-account condition so a source-IP-only spray of many usernames is still visible as spray (alert #3 stays a different problem) while the live Path 2 burst (one attempted account) still qualifies. Lever 1 alone does *not* kill the VPN FP, because that FP is also one username. Lever 2 is the actual suppressor: a Tier 1 SOP that a 100020 fire is not escalated if the same source and account produce a successful SSH login in the next two minutes. Wazuh cannot cleanly say “not followed by success” in XML, so that check is written into the playbook. Analysts close the VPN-retry class (alerts like #5) and still escalate a burst with no success.

```
<group name="meridian,ssh_bruteforce,attack,">
  <rule id="100020" level="10" frequency="5" timeframe="120">
    <if_matched_sid>5760</if_matched_sid>
    <same_source_ip />
    <same_field>dstuser</same_field>
    <description>Meridian: 5+ SSH failures, same source and same username, in 2 minutes</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
    <group>ssh_bruteforce,attack_credential_access,</group>
  </rule>
</group>
```

Tier 1 SOP (appended to the rule). On every 100020 alert, search the next 120 seconds for a successful SSH for that source IP and username. If success is present and the account is a known staff or service identity, close as VPN/job retry and do not escalate. If there is no success, or the account is unused/root/unknown, escalate to Tier 2.

Evidence #7 is Discover showing the benign retry-then-success events with no 100020 escalate (SOP close). Evidence #8 is the original no-success burst still raising 100020.

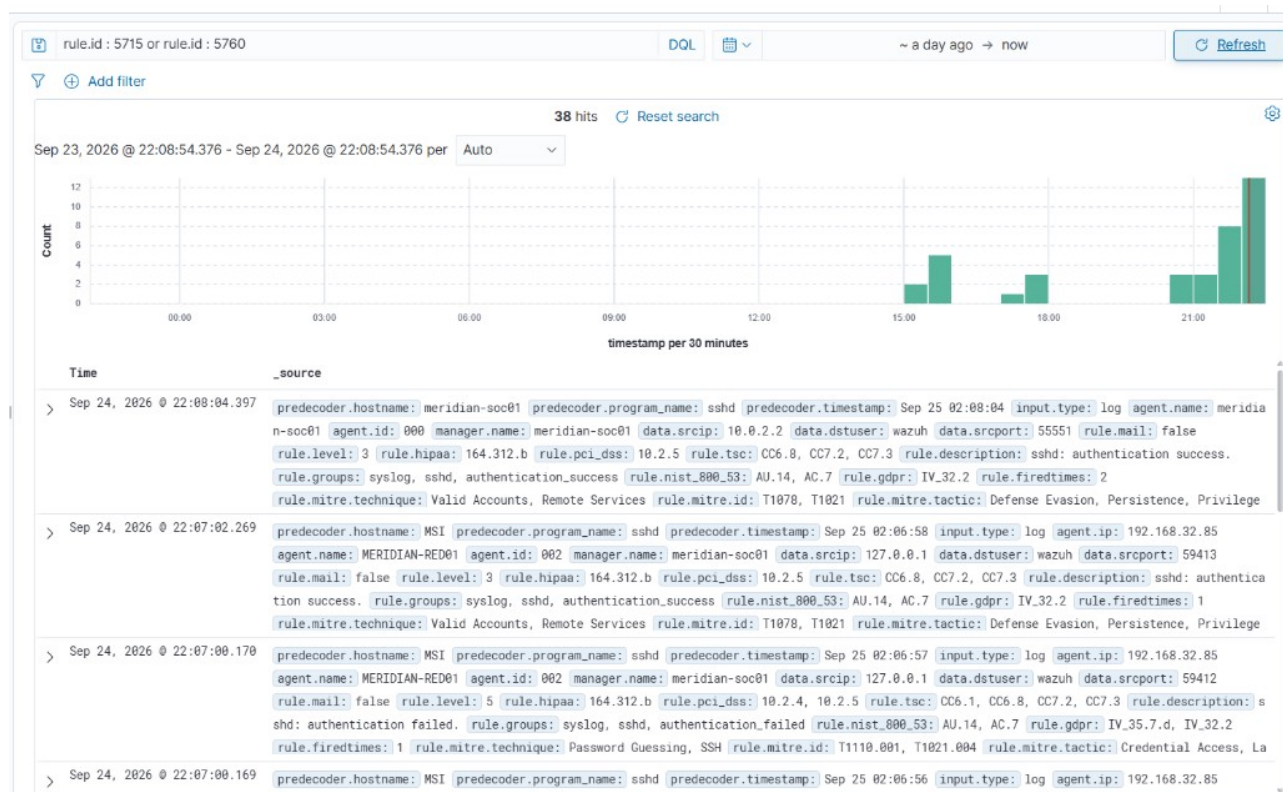


Figure 7: Evidence #7. Tuned path: MERIDIAN-RED01 shows 5760 failures then 5715 success for wazuh from 127.0.0.1. SOP close; no new 100020 escalate.



Figure 8: Evidence #8. Same 100020 still fires on the Part C no-success burst: five Failed password lines, source 127.0.0.1, level 10.

Critical thinking #4. Leaving 100020 untuned spends Tier 1 time, not attacker time: several VPN retries a week become standing tickets, which is the alert-fatigue cost from lecture. Those minutes are taken from the queue that should have been looking at alerts #1, #2, and #6. If Meridian changes VPN concentrators or jump-host key behavior in six months, the SOP and `dstuser` assumption can go stale. I

would keep a small replay set (one recorded benign retry-then-success fixture and one no-success burst) and re-run it after the VPN change, plus a two-week watch on 100020 volume, before calling the tune still valid.